

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory work No. 5:
Assembly Arithmetic &
BCD Number Representation
in NASM

Elaborated:

st. gr. FAF-243 Kushnirenko Ecaterina

Verified:

asist. univ. Fiștic Cristofor

Chișinău - 2026

Contents

1	INTRODUCTION	3
1.1	Registers and Arithmetic in x86 Assembly	3
1.2	BCD (Binary Coded Decimal) Representation	3
2	PURPOSE OF THE REPORT	3
2.1	Variant Selection	4
3	KEY INSTRUCTIONS USED	5
3.1	The DAA Instruction	5
4	TASK 1 — REGISTER EXPRESSION	6
4.1	Formula and Values	6
4.2	Register Usage	6
4.3	Source Code	6
4.4	Step-by-Step Trace	7
5	TASK 2 — ARITHMETIC EXPRESSIONS	8
5.1	Expression 14	8
5.1.1	Formula and Manual Verification	8
5.1.2	Register Usage	8
5.1.3	Source Code	8
5.2	Expression 8	10
5.2.1	Formula and Manual Verification	10
5.2.2	Register Usage	10
5.2.3	Source Code	10
5.3	Expression 10	12
5.3.1	Formula and Manual Verification	12
5.3.2	Register Usage	12
5.3.3	Source Code	12
6	TASK 3 — BCD MANUAL CONVERSIONS (VARIANT 14, EVEN COLUMN)	14
6.1	Decimal to 8421 BCD	14
6.2	Straight Binary vs. BCD — Bit Count	14
6.3	BCD to Decimal Conversion	14
6.4	Adding BCD Binary Strings	14
6.5	Decimal Pairs to BCD and Addition	16
6.6	BCD Subtraction via 9's Complement	16
6.6.1	Method	16
6.6.2	a) $974 - 632 = 342$	16
6.6.3	b) $453 - 983 = -530$	16
6.6.4	Large numbers: $X = 72345$, $Y = -24341$, $X + Y$	17
7	TASK 4 — BCD ADDER CIRCUIT IN LOGISIM	18
7.1	Circuit Description	18
7.2	Truth Table for Correction Signal	18
8	TASK 5 — ASSEMBLY: BCD ADDITION	19
8.1	Description	19
8.2	Source Code	19

8.3	Memory and Register Analysis	20
9	DEBUGGING	21
9.1	Debugging Approach	21
10	CONCLUSION	22

1 INTRODUCTION

1.1 Registers and Arithmetic in x86 Assembly

In x86 Assembly Language, all computation takes place directly in CPU registers — small, ultra-fast storage locations built into the processor. Unlike high-level languages that hide arithmetic behind operators, Assembly requires the programmer to explicitly load operands into registers, apply an instruction, and store the result. This gives complete visibility into every step of a computation.

The general-purpose registers used for arithmetic in 32-bit (IA-32) mode are:

Table 1: General-purpose registers used in this laboratory

Register	Primary Role
EAX	Accumulator — holds operands and results; <code>idiv</code> places quotient here
EBX	General operand storage
ECX	Counter and temporary operand
EDX	Extended accumulator — <code>cdq</code> sign-extends EAX into EDX:EAX before division; <code>idiv</code> places remainder here
ESI	Save register — used to preserve intermediate results across operations

1.2 BCD (Binary Coded Decimal) Representation

Binary Coded Decimal (BCD) is a class of encodings in which each decimal digit (0–9) is represented individually by its binary equivalent. The most common variant is the **8421 BCD** code, where each digit occupies exactly four bits (a nibble), weighted $2^3, 2^2, 2^1, 2^0$.

Table 2: 8421 BCD encoding table

Decimal	BCD	Decimal	BCD
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

The six bit patterns 1010–1111 are invalid in BCD. When arithmetic produces these patterns, a correction of +6 (0110) is applied to skip over the invalid states and return to a valid BCD digit.

2 PURPOSE OF THE REPORT

This report documents Laboratory Work No. 5, Variant 14. The objectives are:

- Write a NASM program that computes the register expression $A = (A + B) - (C + D)$
- Implement three arithmetic expressions selected according to the variant rules
- Perform manual BCD conversions, additions, and subtractions using 9’s complement

- Understand the structure of a BCD adder circuit
- Write a NASM program that adds two packed 8-bit BCD numbers using the DAA instruction

2.1 Variant Selection

Variant number: **14** (even).

- **Expression 1:** Variant 14 $\Rightarrow$ **Expression 14**
- **Expression 2:** 14 is even $\Rightarrow$ add 2: $14 + 2 = 16 > 15$, so divide by 2: $16 \div 2 = 8 \Rightarrow$ **Expression 8**
- **Expression 3:** 8 is even $\Rightarrow$ add 2: $8 + 2 = 10 \leq 15 \Rightarrow$ **Expression 10**
- **BCD section:** Variant 14 is even $\Rightarrow$ use the *even* column numbers: 21, 25, 36, 98, 125, 156

3 KEY INSTRUCTIONS USED

Table 3: NASM instructions used in this laboratory

Instruction	Description
<code>MOV dest, src</code>	Copies a value from source to destination register or memory
<code>ADD dest, src</code>	Adds source to destination; result stored in destination
<code>SUB dest, src</code>	Subtracts source from destination
<code>IMUL dest, src</code>	Signed integer multiplication
<code>CDQ</code>	Sign-extends EAX into the EDX:EAX pair; required before <code>IDIV</code>
<code>IDIV src</code>	Signed division of EDX:EAX by src; quotient $\rightarrow$ EAX, remainder $\rightarrow$ EDX
<code>DAA</code>	Decimal Adjust AL after Addition; corrects AL to a valid packed BCD byte after a binary <code>ADD</code> (32-bit mode only)
<code>JNC label</code>	Jump if Carry Flag = 0 (no carry generated)
<code>XOR reg, reg</code>	Zeros a register efficiently ($r \oplus r = 0$)
<code>INT 0x80</code>	Linux 32-bit syscall interrupt
<code>SYSCALL</code>	Linux 64-bit syscall instruction

3.1 The DAA Instruction

DAA (Decimal Adjust AL after Addition) is a special x86 instruction that corrects the binary result of an `ADD` on two packed BCD bytes. Its internal logic is:

1. If the low nibble of AL exceeds 9, or the Auxiliary Carry flag (AF) is set: `AL += 6`
2. If AL exceeds `0x99`, or the Carry flag (CF) is set: `AL += 0x60`; set CF

Important: DAA is only valid in 32-bit (IA-32) mode. Intel removed it from the x86-64 instruction set. Code using DAA must therefore be assembled with `nasm -f elf32` and linked with `ld -m elf_i386`.

4 TASK 1 — REGISTER EXPRESSION

4.1 Formula and Values

$$A = (A + B) - (C + D)$$

Test values: $A = 10$, $B = 5$, $C = 3$, $D = 2$.

Expected result: $(10 + 5) - (3 + 2) = 15 - 5 = 10$.

4.2 Register Usage

Table 4: Register usage — Task 1

Register	Contents
EAX	Holds $A + B = 15$, then the final result = 10
EBX	Holds $C + D = 5$

4.3 Source Code

```

1 ; Task 1: A = (A + B) - (C + D)
2 ; Works in SASM (64-bit mode, entry point: main)
3
4 section .data
5     A      dd 10
6     B      dd 5
7     C      dd 3
8     D      dd 2
9     result dd 0
10
11 section .text
12     global main
13
14 main:
15     mov eax, [A]      ; eax = A
16     add eax, [B]      ; eax = A + B
17
18     mov ebx, [C]      ; ebx = C
19     add ebx, [D]      ; ebx = C + D
20
21     sub eax, ebx      ; eax = (A+B) - (C+D)
22
23     mov [result], eax ; store result
24
25     xor eax, eax      ; return 0
26     ret

```

Listing 1: task1_expression.asm — $A = (A+B) - (C+D)$

4.4 Step-by-Step Trace

Table 5: Execution trace — Task 1

Instruction	EAX	EBX
mov eax, [A]	10	—
add eax, [B]	15	—
mov ebx, [C]	15	3
add ebx, [D]	15	5
sub eax, ebx	10	5

5 TASK 2 — ARITHMETIC EXPRESSIONS

5.1 Expression 14

5.1.1 Formula and Manual Verification

$$z = \frac{a \cdot c - b \cdot d}{f} + \frac{(-a + b) \cdot c}{d} \Big/ h$$

Values: $a = 6$, $b = 3$, $c = 9$, $d = 3$, $f = 3$, $h = 2$.

$$z = \frac{6 \cdot 9 - 3 \cdot 3}{3} + \frac{(-6 + 3) \cdot 9}{3} \Big/ 2 = \frac{45}{3} + \frac{-27}{3} \Big/ 2 = \frac{15 + (-9)}{2} = \frac{6}{2} = 3$$

5.1.2 Register Usage

Table 6: Register usage — Expression 14

Register	Contents at key steps
EAX	$a \cdot c = 54 \rightarrow 54 - 9 = 45 \rightarrow 45/3 = 15 \rightarrow b - a = -3 \rightarrow -3 \cdot c = -27 \rightarrow -27/d = -9 \rightarrow -9 + 15 = 6 \rightarrow 6/2 = 3$
EBX	$b \cdot d = 9$
ECX	First term: $(a \cdot c - b \cdot d)/f = 15$ (saved across part 2)
EDX	Sign-extension half of dividend (set by <code>cdq</code>)

5.1.3 Source Code

```

1 ; Task 2 - Expression 14: z = ((a*c - b*d)/f + (-a+b)*c/d) / h
2 ; Values: a=6, b=3, c=9, d=3, f=3, h=2 => z = 3
3 ; Works in SASM (64-bit mode, entry point: main)
4
5 section .data
6     a      dd    6
7     b      dd    3
8     c      dd    9
9     d      dd    3
10    f      dd    3
11    h      dd    2
12    result dd    0
13
14 section .text
15     global main
16
17 main:
18     ; --- part 1: (a*c - b*d) / f ---
19     mov     eax, [a]
20     imul    eax, dword [c]      ; eax = a*c = 54
21     mov     ebx, [b]
22     imul    ebx, dword [d]      ; ebx = b*d = 9
23     sub     eax, ebx           ; eax = 45
24     cdq
25     idiv    dword [f]          ; eax = 45/3 = 15
26     mov     ecx, eax           ; save first term
27
28     ; --- part 2: (-a+b)*c / d ---

```

```
29  mov    eax, [b]
30  sub    eax, [a]          ; eax = b-a = -3
31  imul   eax, dword [c]    ; eax = -3*9 = -27
32  cdq
33  idiv   dword [d]         ; eax = -27/3 = -9
34
35  ; --- part 3: add terms, divide by h ---
36  add    eax, ecx          ; eax = -9+15 = 6
37  cdq
38  idiv   dword [h]         ; eax = 6/2 = 3 => z
39
40  mov    [result], eax
41
42  xor    eax, eax
43  ret
```

Listing 2: task2_expr14.asm — Expression 14

5.2 Expression 8

5.2.1 Formula and Manual Verification

$$z = \frac{(-a + b + c + 1)^3}{(a - b \cdot c + d)^2}$$

Values: $a = 2$, $b = 3$, $c = 1$, $d = 2$.

$$\text{num base} = -2 + 3 + 1 + 1 = 3 \Rightarrow 3^3 = 27$$

$$\text{den base} = 2 - 3 \cdot 1 + 2 = 1 \Rightarrow 1^2 = 1$$

$$z = 27/1 = \mathbf{27}$$

5.2.2 Register Usage

Table 7: Register usage — Expression 8

Register	Contents at key steps
EAX	base = 3 $\rightarrow$ base ² = 9 $\rightarrow$ base ³ = 27 (numerator) $\rightarrow z = 27$
EBX	base = 3 (saved for repeated multiplication)
ECX	numerator = 27
EDX	denominator base = 1 $\rightarrow$ squared = 1

5.2.3 Source Code

```

1 ; Task 2 - Expression 8: z = (-a+b+c+1)^3 / (a - b*c + d)^2
2 ; Values: a=2, b=3, c=1, d=2 => z = 27
3 ; nasm -f elf64 task2_expr8.asm -o task2_expr8.o && ld task2_expr8.o -o
   task2_expr8
4 ; Works in SASM (64-bit mode, entry point: main)
5
6 section .data
7     a      dd    2
8     b      dd    3
9     c      dd    1
10    d      dd    2
11    result dd    0
12
13 section .text
14     global main
15
16 main:
17     ; --- numerator base: -a+b+c+1 ---
18     mov     eax, [b]
19     sub     eax, [a]      ; eax = b-a = 1
20     add     eax, [c]      ; eax = 2
21     add     eax, 1        ; eax = 3
22
23     ; --- base^3 ---
24     mov     ebx, eax      ; save base
25     imul    eax, ebx      ; eax = 9
26     imul    eax, ebx      ; eax = 27 (numerator)
27     mov     ecx, eax
28

```

```

29     ; --- denominator base: a - b*c + d ---
30     mov     eax, [b]
31     imul    eax, dword [c]      ; eax = b*c = 3
32     mov     edx, [a]
33     sub     edx, eax           ; edx = a - b*c = -1
34     add     edx, [d]           ; edx = 1
35
36     ; --- base^2 ---
37     imul    edx, edx           ; edx = 1 (denominator)
38
39     ; --- z = numerator / denominator ---
40     mov     edi, edx           ; save denominator before cdq
41     mov     eax, ecx           ; eax = 27 (numerator)
42     cdq
43     idiv    edi               ; eax = 27/1 = 27 => z
44
45     mov     [result], eax
46
47     xor     eax, eax
48     ret

```

Listing 3: task2_expr8.asm — Expression 8

5.3 Expression 10

5.3.1 Formula and Manual Verification

$$z = \frac{-a^2 + b^2}{a^2 - b^2 - 5}$$

Values: $a = 5$, $b = 3$.

numerator = $-25 + 9 = -16$ denominator = $25 - 9 - 5 = 11$

$z = -16 \div 11 = -1$ (integer division, truncates toward zero)

5.3.2 Register Usage

Table 8: Register usage — Expression 10

Register	Contents at key steps
EAX	$a^2 = 25$ (temp) $\rightarrow$ numerator = $-16 \rightarrow z = -1$
EBX	$b^2 = 9$
ECX	numerator = -16
EDX	denominator = 11
ESI	$a^2 = 25$ (saved for reuse in both numerator and denominator)

5.3.3 Source Code

```

1 ; Task 2 - Expression 10: z = (-a^2 + b^2) / (a^2 - b^2 - 5)
2 ; Values: a=5, b=3 => z = -1
3 ; nasm -f elf64 task2_expr10.asm -o task2_expr10.o && ld task2_expr10.o -o
   task2_expr10
4 ; Works in SASM (64-bit mode, entry point: main)
5
6 section .data
7     a      dd    5
8     b      dd    3
9     result dd    0
10
11 section .text
12     global main
13
14 main:
15     ; --- a^2 ---
16     mov     eax, [a]
17     imul    eax, eax           ; eax = 25
18     mov     esi, eax           ; save a^2
19
20     ; --- b^2 ---
21     mov     ebx, [b]
22     imul    ebx, ebx           ; ebx = 9
23
24     ; --- numerator: -a^2 + b^2 ---
25     mov     ecx, ebx
26     sub     ecx, esi           ; ecx = 9-25 = -16
27
28     ; --- denominator: a^2 - b^2 - 5 ---
29     mov     edx, esi

```

```
30     sub    edx, ebx           ; edx = 16
31     sub    edx, 5             ; edx = 11
32
33     ; --- z = numerator / denominator ---
34     mov    eax, ecx
35     cdq
36     idiv   edx                ; eax = -16/11 = -1 => z
37
38     mov    [result], eax
39
40     xor    eax, eax
41     ret
```

Listing 4: task2_expr10.asm — Expression 10

6 TASK 3 — BCD MANUAL CONVERSIONS (VARIANT 14, EVEN COLUMN)

Even variant $\Rightarrow$ numbers from the even column: **21, 25, 36, 98, 125, 156**.

6.1 Decimal to 8421 BCD

Table 9: Decimal to BCD conversion

Decimal	Digit breakdown	8421 BCD
21	2 = 0010, 1 = 0001	0010 0001
25	2 = 0010, 5 = 0101	0010 0101
36	3 = 0011, 6 = 0110	0011 0110
98	9 = 1001, 8 = 1000	1001 1000
125	1 = 0001, 2 = 0010, 5 = 0101	0001 0010 0101
156	1 = 0001, 5 = 0101, 6 = 0110	0001 0101 0110

6.2 Straight Binary vs. BCD — Bit Count

Table 10: Binary bits vs. BCD bits required

Decimal	Binary	Bits	BCD	BCD bits	Extra
21	10101	5	0010 0001	8	+3
25	11001	5	0010 0101	8	+3
36	100100	6	0011 0110	8	+2
98	1100010	7	1001 1000	8	+1
125	1111101	7	0001 0010 0101	12	+5
156	10011100	8	0001 0101 0110	12	+4

BCD always requires more bits than straight binary because six of the sixteen possible nibble values (1010–1111) are unused, wasting storage.

6.3 BCD to Decimal Conversion

Table 11: BCD to decimal

BCD string	Split into nibbles	Decimal
001000110111	0010 0011 0111 $\rightarrow$ 2, 3, 7	237
010000100001	0100 0010 0001 $\rightarrow$ 4, 2, 1	421
100000000000	1000 0000 0000 $\rightarrow$ 8, 0, 0	800

6.4 Adding BCD Binary Strings

(f) **01100100 + 00110011** (decimal: $64 + 33 = 97$)

```

  01100100    (6, 4)
+ 00110011    (3, 3)
-----
 10010111    -> nibbles: 1001 | 0111 = 9, 7 -> 97 (no correction needed)
```

(h) **10000101 + 00010011** (decimal: $85 + 13 = 98$)

```

10000101    (8, 5)
+ 00010011    (1, 3)
-----
10011000    -> nibbles: 1001 | 1000 = 9, 8 -> 98 (both <= 9, valid)

```

(e) **00100101 + 00100111** (decimal: $25 + 27 = 52$)

```

Ones nibble: 0101 + 0111 = 1100 (12 > 9) -> INVALID
              add 6: 1100 + 0110 = 0010, carry = 1
Tens nibble: 0010 + 0010 + 1(carry) = 0101 = 5 (valid)
Result: 0101 0010 -> 52 (correct)

```

(g) **10011000 + 10010111** (decimal: $98 + 97 = 195$)

```

Ones nibble: 1000 + 0111 = 1111 (15 > 9) -> add 6 -> 0101, carry=1
Tens nibble: 1001 + 1001 + 1 = 10011 -> add 6 -> 1001, carry=1
Hundreds:    0001
Result: 0001 1001 0101 -> 195 (correct)

```

6.5 Decimal Pairs to BCD and Addition

a) $421 + 975 = 1396$

421 in BCD	0100 0010 0001
975 in BCD	1001 0111 0101
Ones: $0001 + 0101$	$= 0110 = 6 \leq 9$ (valid)
Tens: $0010 + 0111$	$= 1001 = 9 \leq 9$ (valid)
Hundreds: $0100 + 1001$	$= 1101 = 13 > 9 \rightarrow$ add 6: carry=1, digit=3
Thousands: $0 + 0 + 1$	$= 0001$
Result BCD	0001 0011 1001 0110 $\rightarrow$ 1396

b) $183 + 195 = 378$

183 in BCD	0001 1000 0011
195 in BCD	0001 1001 0101
Ones: $0011 + 0101$	$= 1000 = 8 \leq 9$ (valid)
Tens: $1000 + 1001$	$= 10001 > 9 \rightarrow$ add 6 $\rightarrow$ 0111, carry=1
Hundreds: $0001 + 0001 + 1$	$= 0011 = 3$ (valid)
Result BCD	0011 0111 1000 $\rightarrow$ 378

6.6 BCD Subtraction via 9's Complement

6.6.1 Method

To compute $A - B$ in BCD:

1. Find the 9's complement of B : subtract each decimal digit from 9.
2. Add A and the 9's complement of B using BCD addition rules.
3. If a carry is generated (end-around carry), add it to the result. That carry signals a positive result.
4. If no carry, the result is negative; take the 9's complement of the sum.

6.6.2 a) $974 - 632 = 342$

9's complement of 632: $9 - 6 = 3$, $9 - 3 = 6$, $9 - 2 = 7 \Rightarrow$ **367**

$974 + 367$ (BCD)

Ones: $0100 + 0111 = 1011 > 9 \rightarrow$ add 6 $\rightarrow$ 0001, carry=1

Tens: $0111 + 0110 + 1 = 1110 > 9 \rightarrow$ add 6 $\rightarrow$ 0100, carry=1

Hundreds: $1001 + 0011 + 1 = 1101 > 9 \rightarrow$ add 6 $\rightarrow$ 0011, carry=1 (end-around)

Sum = $0341 +$ end-around carry 1 = 342 $\rightarrow$ POSITIVE (correct)

6.6.3 b) $453 - 983 = -530$

9's complement of 983: $9 - 9 = 0$, $9 - 8 = 1$, $9 - 3 = 6 \Rightarrow$ **016**

$453 + 016$ (BCD)

Ones: $0011 + 0110 = 1001 = 9$ (valid)

Tens: $0101 + 0001 = 0110 = 6$ (valid)

Hundreds: $0100 + 0000 = 0100 = 4$ (valid)
 Sum = 469, NO carry $\rightarrow$ NEGATIVE
 9's complement of 469: $9-4=5$, $9-6=3$, $9-9=0 \rightarrow 530$
 Result: -530 (correct)

6.6.4 Large numbers: $X = 72345$, $Y = -24341$, $X + Y$

Compute $72345 - 24341$ using 5-digit 9's complement.

9's complement of 24341: $99999 - 24341 = 75658$

72345 + 75658 (BCD):

Ones: $5+8=13 > 9 \rightarrow$ add 6 $\rightarrow 3$, carry=1

Tens: $4+5+1=10 > 9 \rightarrow$ add 6 $\rightarrow 0$, carry=1

Hundreds: $3+6+1=10 > 9 \rightarrow$ add 6 $\rightarrow 0$, carry=1

Thousands: $2+5+1=8$ (valid)

Ten-thousands: $7+7=14 > 9 \rightarrow$ add 6 $\rightarrow 4$, carry=1 (end-around)

Sum = 48003 + 1 = 48004 $\rightarrow$ POSITIVE

$72345 + (-24341) = 48004$ (correct)

7 TASK 4 — BCD ADDER CIRCUIT IN LOGISIM

7.1 Circuit Description

A single-digit BCD adder takes two 4-bit BCD digits ($A_{3:0}$ and $B_{3:0}$) and a carry-in (C_{in}), and produces a 4-bit BCD sum digit plus a carry-out (C_{out}).

The circuit consists of two main stages:

1. **Binary addition:** A standard 4-bit binary full-adder (FA) adds $A_{3:0}$, $B_{3:0}$, and C_{in} . The raw 4-bit result $S_{3:0}$ and the carry-out C_4 are produced.
2. **Correction logic:** The correction (add 6 = 0110) is needed when the raw sum is invalid BCD, i.e.

$$\text{CORRECT} = C_4 + (S_3 \cdot S_2) + (S_3 \cdot S_1)$$

If $\text{CORRECT} = 1$, a second 4-bit adder adds 0110 to $S_{3:0}$. Any carry from this second addition becomes the BCD C_{out} .

Table 12: BCD adder — Logisim component list

Component	Purpose
4-bit binary adder ($\times 2$)	First adds the BCD digits; second optionally adds the correction 0110
3-input OR gate	Computes $\text{CORRECT} = C_4 + S_3S_2 + S_3S_1$
2-input AND gate ($\times 2$)	Computes $S_3 \cdot S_2$ and $S_3 \cdot S_1$
4-bit MUX / AND gates	Selects 0110 as second operand of second adder when $\text{CORRECT} = 1$
OR gate for C_{out}	Combines carry from second adder with CORRECT signal

7.2 Truth Table for Correction Signal

Table 13: When correction is applied (selected cases)

C_4	S_3	S_2	S_1	CORRECT	Action
0	0	x	x	0	No correction
0	1	0	0	0	No correction (sum = 8 or 9)
0	1	0	1	1	Add 6 (sum = 10 or 11)
0	1	1	x	1	Add 6 (sum = 12–15)
1	x	x	x	1	Add 6 (carry means sum ≥ 16)

8 TASK 5 — ASSEMBLY: BCD ADDITION

8.1 Description

The program adds two packed 8-bit BCD numbers and stores the two-byte result (low digit byte and carry byte) in sequential memory locations. Because the DAA instruction is unavailable in 64-bit mode, the code is assembled as a 32-bit ELF binary.

- **Build:** `nasm -f elf32 task5_bcd_add.asm -o task5_bcd_add.o`
- **Link:** `ld -m elf_i386 task5_bcd_add.o -o task5_bcd_add`
- **Note:** On a 64-bit system you may need `sudo apt install gcc-multilib` (Debian/Ubuntu) or `sudo pacman -S lib32-glibc` (Arch Linux) for 32-bit support.

Example: BCD $0x47$ (= 47) + BCD $0x38$ (= 38)

1. Binary sum: $0x47 + 0x38 = 0x7F$
2. DAA: lower nibble $F = 15 > 9 \Rightarrow AL += 6$: $0x7F + 0x06 = 0x85$
3. Upper nibble $= 8 \leq 9$ — no second correction; $CF = 0$
4. Result: $AL = 0x85$ (BCD 85), carry = 0

8.2 Source Code

```

1 ; Task 5: Add two 8-bit packed BCD numbers
2 ; DAA only works in 32-bit mode!
3 ; In SASM: Settings -> Assembler -> check "Use 32-bit program"
4 ;
5 ; Example: 0x47 + 0x38 = 0x85 (47 + 38 = 85), carry = 0
6 ;
7 section .data
8     bcd1    db 0x47                ; first BCD number (47)
9     bcd2    db 0x38                ; second BCD number (38)
10
11 section .bss
12     result  resb 1                ; BCD result byte
13     carry_out resb 1              ; carry byte (0x00 or 0x01)
14
15 section .text
16     global main
17
18 main:
19     mov     al, [bcd1]             ; al = 0x47
20     add     al, [bcd2]             ; al = 0x7F (raw binary, not valid BCD yet)
21     daa                                ; fix al to valid BCD: 0x7F -> 0x85
22                                ; DAA adds 6 to any nibble that exceeds 9
23
24     mov     [result], al          ; store result
25
26     jnc     no_carry
27     mov     byte [carry_out], 0x01 ; result > 99, carry to next digit
28     jmp     done
29
30 no_carry:

```

```

31     mov     byte [carry_out], 0x00
32
33 done:
34     xor     eax, eax
35     ret

```

Listing 5: task5_bcd_add.asm — Add two 8-bit BCD numbers

8.3 Memory and Register Analysis

Table 14: Memory state before and after execution

Location	Before	After	Meaning
[bcd1]	0x47	0x47	BCD 47 (input, unchanged)
[bcd2]	0x38	0x38	BCD 38 (input, unchanged)
[result]	0x00	0x85	BCD 85 (packed: tens=8, ones=5)
[carry_out]	0x00	0x00	No carry (result ≤ 99)

Table 15: Register trace during execution

After instruction	AL value and flags
mov al, [bcd1]	AL = 0x47
add al, [bcd2]	AL = 0x7F (AF=1 because lower nibble 7+8=15 > 9)
daa — lower check	Nibble F = 15 > 9 $\rightarrow$ AL += 6: 0x7F + 0x06 = 0x85
daa — upper check	Nibble 8 $\leq$ 9 $\rightarrow$ no correction; CF = 0
mov [result], al	[result] = 0x85 (BCD 85)
jnc no_carry	CF = 0 $\rightarrow$ jump taken
mov byte [carry_out], 0x00	[carry_out] = 0x00

9 DEBUGGING

Debugging Assembly code requires careful attention to register and memory state at every step. The following issues were encountered during development:

Table 16: Debugging issues and solutions

Issue	Cause	Solution
IDIV produced wrong quotient or fault	CDQ was missing before IDIV; EDX contained leftover data from a previous multiply	Add <code>cdq</code> immediately before every <code>idiv</code> to sign-extend EAX into EDX:EAX
Expression 8: <code>idiv edx</code> crashed (division by zero)	After <code>cdq</code> , EDX is overwritten; the denominator stored in EDX was destroyed	Compute and save the denominator in EDX <i>before</i> calling <code>cdq/idiv</code>
Task 5 assembled for 64-bit (elf64)	DAA is not supported in x86-64 mode; NASM gave an “instruction not supported in 64-bit mode” error	Switched to <code>nasm -f elf32</code> and <code>ld -m elf_i386</code>
<code>IMUL ebx, dword [d]</code> used with a memory operand	Two-operand <code>IMUL</code> requires the destination to be a register; using a memory operand gives an assemble error	Load memory value into a register first, then <code>IMUL</code>

9.1 Debugging Approach

All programs were debugged using **GDB** with the following workflow:

1. Assemble with debug symbols: add the `-g` flag to NASM.
2. Start GDB: `gdb ./task2_expr14`
3. Set breakpoint at the division instruction: `break *_start+N`
4. Run: `run`
5. Inspect registers: `info registers eax ebx ecx edx`
6. Single-step: `stepi` (one instruction at a time)
7. Check memory: `x/4xb &result`

This approach made it easy to verify intermediate values (e.g. confirming `EAX = 45` before the first `idiv` in Expression 14) and to catch the missing `cdq` bug early.

10 CONCLUSION

This laboratory covered two complementary topics in low-level computing: register-based arithmetic in NASM Assembly and the BCD (Binary Coded Decimal) number system.

Through Task 1 and Task 2, the mechanics of signed integer arithmetic using `IMUL`, `IDIV`, and `CDQ` were practised. A key lesson was that `CDQ` must always precede `IDIV`, and that intermediate results must be saved in registers such as `ECX` or `ESI` before they are overwritten by subsequent operations.

The BCD section (Task 3) demonstrated why BCD is used in contexts where exact decimal representation matters (e.g. financial software, seven-segment displays): each decimal digit is stored independently, so no rounding errors or base-conversion artefacts occur. The manual addition and 9's complement subtraction exercises illustrated how the six invalid states (1010–1111) necessitate the +6 correction step.

Task 4 showed how the correction logic can be implemented in hardware using two 4-bit adders and a small combinational circuit. Task 5 demonstrated the software equivalent via the `DAA` instruction, which encapsulates exactly the same correction rules in a single opcode. The important architectural note is that `DAA` was removed in x86-64, requiring 32-bit mode for any code that depends on it.

Overall, the laboratory strengthened understanding of how computers represent and manipulate numbers at the hardware level, and highlighted the gap between the abstractions provided by high-level languages and the actual operations performed by the processor.